Projected Coordinate SBM

1 Introduction

...

2 Related Work

...

3 Method

3.1 Model

Consider a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with node set $\mathcal{V}$ and edge set $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$. The associated adjacency matrix $\mathbf{A}$ is defined such that $a_{uv} = 1$ if $(u, v) \in \mathcal{E}$ and 0 otherwise. We assume that $\mathcal{G}$ is generated by a Stochastic Block Model (SBM), so that each node $u \in \mathcal{V}$ belongs to one of k communities. While there are many forms of the SBM designed to address different assumptions, we restrict our focus to the standard Bernoulli parameterization.

Community memberships in the SBM are given by a partition matrix $\mathbf{Z}$, which is defined such that $z_{ui} = 1$ if node u is a member of community i and 0 otherwise. We assume that $\mathbf{z}_u \sim \text{Multinomial}(1, \boldsymbol{\pi})$ where π_i is the probability of any node belonging to community i. The SBM is also parameterized by a structure matrix $\boldsymbol{\Theta}$, which is defined such that θ_{ij} is the probability that an edge exists from a node in community i to a node and community j. That is, each element of the adjacency matrix is independently and identically distributed according to $a_{uv}|\mathbf{Z} \sim \text{Bernoulli}(\mathbf{z}_u'\boldsymbol{\Theta}\mathbf{z}_v)$.

The complete log-likelihood of this model is

$$\ell^*(\mathbf{Z}, \boldsymbol{\pi}, \boldsymbol{\Theta}|\mathbf{A}) = \sum_{u,v} \left[a_{uv} \ln\left(\mathbf{z}_u'\boldsymbol{\Theta}\mathbf{z}_v\right) + (1-a_{uv}) \ln\left(1-\mathbf{z}_u'\boldsymbol{\Theta}\mathbf{z}_v\right)\right] + \sum_u \ln\left(\mathbf{z}_u'\boldsymbol{\pi}\right) \tag{1}$$

where the first summation is over every dyad $(u, v) \in \mathcal{V} \times \mathcal{V}$ and the second summation is over every node $u \in \mathcal{V}$. For the most part, our focus in this paper is on the log-likelihood of the dyads alone. Therefore we instead consider the expression

$$\ell(\mathbf{Z}, \boldsymbol{\Theta}|\mathbf{A}) = \sum_{u,v} \left[a_{uv} \ln(p_{uv}) + (1-a_{uv}) \ln(1-p_{uv})\right] \tag{2}$$

where $p_{uv} = \mathbf{z}_u' \mathbf{\Theta} \mathbf{z}_v$ for ease of notation. Differentiating this log-likelihood with respect to one row of $\mathbf{Z}$ gives

$$\begin{aligned} \frac{\partial \ell}{\partial \mathbf{z}_u} &= \sum_v \left[\frac{a_{uv} - p_{uv}}{p_{uv}(1-p_{uv})} \right] \mathbf{\Theta} \mathbf{z}_v \qquad (3) \\ &= \mathbf{\Theta}' \mathbf{Z}' \mathbf{W}_u (\mathbf{a}_u - \mathbf{p}_u) \\ \frac{\partial \ell^2}{\partial \mathbf{z}_u \partial \mathbf{z}_u'} &= -\sum_v \left[\frac{a_{uv} - p_{uv}}{p_{uv}(1-p_{uv})} \right]^2 \mathbf{\Theta} \mathbf{z}_v \mathbf{z}_v' \mathbf{\Theta}' \qquad (4) \\ &= -\mathbf{\Theta}' \mathbf{Z}' \mathbf{W}_u \mathbf{S}_u \mathbf{W}_u \mathbf{Z} \mathbf{\Theta} \end{aligned}$$

where $\mathbf{W}_u$ is defined over nodes $v \in \mathcal{V}$ with u fixed as

$$\mathbf{W}_u = \mathbf{W}_u(\mathbf{p}_u) := \operatorname{diag}\left(\dots \frac{1}{p_{uv}(1-p_{uv})} \dots \right). \qquad (5)$$

The Fisher Information with respect to $\mathbf{Z}$ is

$$\begin{aligned} \boldsymbol{\mathcal{I}}(\mathbf{Z}) &= \mathrm{E}\left(\frac{\partial \ell^2}{\partial \mathbf{z}_u \partial \mathbf{z}_u'} \right) \qquad (6) \\ &= -\mathbf{\Theta}' \mathbf{Z}' \mathbf{W}_u \mathbf{Z} \mathbf{\Theta} \end{aligned}$$

It should be noted that the gradient above is inexact, as the case where $u = v$ requires an additional scaling factor. That is, $\frac{dp_{uv}}{d\mathbf{z}_u} = \mathbf{\Theta} \mathbf{z}_v$ if $u \neq v$ and $\frac{dp_{uv}}{d\mathbf{z}_u} = 2\mathbf{\Theta} \mathbf{z}_v$ if $u = v$. This factor is ignored for ease of notation.

3.2 Estimation

To estimate the parameters of this model, we use the closed form maximum likelihood estimate (MLE) for the structure matrix $\mathbf{\Theta}$ and the frequency vector $\boldsymbol{\pi}$. We use an approximate Fisher scoring method for the partition matrix $\mathbf{Z}$.

3.2.1 Structure and Frequency

The MLE of θ_{ij} is simply the average over the adjacency matrix between nodes in community i and community j. Specifically,

$$\hat{\theta}_{ij} = \left[\hat{\mathbf{\Theta}}(\mathbf{A}, \mathbf{Z}) \right]_{ij} := \frac{m_{ij}}{n_i \, n_j} \qquad (7)$$

where $\mathbf{M} = \mathbf{Z}' \mathbf{A} \mathbf{Z}$ and $\mathbf{n} = \sum_u \mathbf{z}_u$.

The MLE of $\boldsymbol{\pi}$ is the average over the rows of the partition matrix. That is,

$$\hat{\boldsymbol{\pi}} = \hat{\boldsymbol{\pi}}(\mathbf{Z}) := \frac{1}{|\mathcal{V}|} \sum_u \mathbf{z}_u. \qquad (8)$$

Although it is not used in our log-likelihood (equation 2), $\hat{\boldsymbol{\pi}}$ will be relevant for the extension of our method in Section 3.4.

3.2.2 Partition

To obtain $\hat{\mathbf{z}}_u$, we iteratively apply Fisher updates to each node u followed by a projection onto the feasible set of $\mathbf{z}_u$. The standard Fisher scoring procedure follows the update rule

$$\mathbf{z}_u^{(t+1)} \leftarrow \mathbf{z}_u^{(t)} - \mathrm{E}\left(\frac{\partial \ell^2}{\partial \mathbf{z}_u \partial \mathbf{z}_u'}\right)^{-1} \left(\frac{\partial \ell}{\partial \mathbf{z}_u}\right)\Bigg|_{\mathbf{z}_u = \mathbf{z}_u^{(t)}} \tag{9}$$

where $\mathbf{z}_u^{(t)}$ denotes the estimate of $\mathbf{z}_u$ after the t^{th} iteration. Now define $\mathbf{X}^{(t)} = \mathbf{Z}^{(t)}\mathbf{\Theta}^{(t)}$ and $\mathbf{p}_u^{(t)} = \mathbf{X}^{(t)}\mathbf{z}_u^{(t)}$ for convenience. Plugging in equation 6, the full update is

$$\begin{aligned} \mathbf{z}_u^{(t+1)} &\leftarrow \mathbf{z}_u^{(t)} + \left(\mathbf{X}^{(t)\prime}\mathbf{W}_u^{(t)}\mathbf{X}^{(t)}\right)^{-1}\mathbf{X}^{(t)\prime}\mathbf{W}_u^{(t)}(\mathbf{a}_u - \mathbf{p}_u^{(t)}) \\ &= \left(\mathbf{X}^{(t)\prime}\mathbf{W}_u^{(t)}\mathbf{X}^{(t)}\right)^{-1}\mathbf{X}^{(t)\prime}\mathbf{W}_u^{(t)}\mathbf{a}_u \end{aligned} \tag{10}$$

where $\mathbf{\Theta}^{(t)} = \hat{\mathbf{\Theta}}(\mathbf{A}, \mathbf{Z}^{(t)})$ is the structure MLE given partition $\mathbf{Z}^{(t)}$ and $\mathbf{W}_u^{(t)} = \mathbf{W}_u(\mathbf{p}_u^{(t)})$ is the diagonal weight matrix.

This approach works well when the variable in question is continuous and unbounded. In our case however, $\mathbf{z}_u$ is restricted to a feasible set of the standard basis vectors — that is, unit vectors with all elements but one equal to zero. For this reason, we project the partition update back onto the feasible set using the "hardmax" function:

$$\tilde{\mathbf{z}}_u^{(t+1)} \leftarrow \left(\mathbf{X}^{(t)\prime}\mathbf{W}_u^{(t)}\mathbf{X}^{(t)}\right)^{-1}\mathbf{X}^{(t)\prime}\mathbf{W}_u^{(t)}\mathbf{a}_u \tag{11}$$

$$\mathbf{z}_u^{(t+1)} \leftarrow \mathrm{hardmax}(\tilde{\mathbf{z}}_u^{(t+1)}) \tag{12}$$

where $\mathrm{hardmax}(\tilde{\mathbf{z}}_u)_i = 1$ if $i = \arg\max_j \tilde{\mathbf{z}}_{uj}$ otherwise 0.

Because graphs often contain many nodes, updating the partition of each node individually can be prohibitively expensive. Thus we propose to update the full partition all at once by defining the average weight matrix

$$\overline{\mathbf{W}} = \frac{1}{|\mathcal{V}|}\sum_u \mathbf{W}_u \tag{13}$$

and applying the update rule

$$\tilde{\mathbf{Z}}^{(t+1)} \leftarrow \left[\left(\mathbf{X}^{(t)\prime}\overline{\mathbf{W}}^{(t)}\mathbf{X}^{(t)}\right)^{-1}\mathbf{X}^{(t)\prime}\overline{\mathbf{W}}^{(t)}\mathbf{A}\right]' \tag{14}$$

$$\mathbf{Z}^{(t+1)} \leftarrow \mathrm{hardmax}(\tilde{\mathbf{Z}}^{(t+1)}) \tag{15}$$

where $\overline{\mathbf{W}}^{(t)} = \overline{\mathbf{W}}(\mathbf{P}^{(t)})$ and hardmax is applied row-wise. This is an approximation that offers a significant increase in computational efficiency. To ensure stability and smoothness, we also add a smoothness regularization term $\alpha\mathbf{I}$ to the shared Fisher matrix, where α is the regularizer strength.

The full algorithm iterates over the following three steps until convergence is reached:

$$i.) \qquad \tilde{\mathbf{Z}}^{(t+1)} \leftarrow \mathbf{A}'\,\overline{\mathbf{W}}^{(t)}\mathbf{X}^{(t)}\left(\mathbf{X}^{(t)\prime}\overline{\mathbf{W}}^{(t)}\mathbf{X}^{(t)} + \alpha\mathbf{I}\right)^{-1} \tag{16a}$$

$$ii.) \qquad \mathbf{Z}^{(t+1)} \leftarrow \mathrm{hardmax}(\tilde{\mathbf{Z}}^{(t+1)}) \tag{16b}$$

$$iii.) \qquad \mathbf{\Theta}^{(t+1)} \leftarrow \hat{\mathbf{\Theta}}(\mathbf{A}, \mathbf{Z}^{(t+1)}) \tag{16c}$$

Convergence is reached when $\mathbf{Z}^{(t)}$ changes by less than ϵ after each update. Some tuning is necessary to identify the optimal number of communities k and ridge strength α.

3.3 Initialization

The partition matrix $\mathbf{Z}^{(0)}$ can be initialized as samples from a Multinomial(1, $\frac{1}{k}$) and the structure matrix can be initialized as $\mathbf{\Theta}^{(0)} = \hat{\mathbf{\Theta}}(\mathbf{A}, \mathbf{Z}^{(0)})$. Better results are achieved by using clustering algorithms instead of random samples for $\mathbf{Z}^{(0)}$. For example, K-Means and agglomerative clustering can be applied to the singular vectors of the adjacency matrix to estimate an initial partition. Alternatively, the Louvain and Label Propagation algorithms can be applied directly to the graph structure to obtain the initial partition.

Furthermore, initializing algorithms like agglomerative clustering or Louvain community detection allows us to identify the number of communities when k is unknown. In the next section, we propose our own approach to dynamically identify k dynamically by modifying the algorithm in Section 3.2.2.

3.4 Unknown k

If the true number of communities k is unknown, we propose to start with a large number of communities and iteratively drop those which are not "active". By "active", we mean communities containing at least a minimum number of nodes, $n_{\min}$. Thus, the initial number of communities is set to $k_0 = \left\lfloor \frac{|\mathcal{V}|}{n_{\min}} \right\rfloor$ and after each update to the partition matrix, communities with fewer than $n_{\min}$ nodes are dropped.

To force elements of $\mathbf{n}$ to shrink toward the threshold $n_{\min}$, we augment the log-likelihood objective with a per-node penalty:

$$\ell_{\text{pen}}(\mathbf{Z}, \mathbf{\Theta}|\mathbf{A}) = \ell(\mathbf{Z}, \mathbf{\Theta}|\mathbf{A}) - \beta \textstyle\sum_u r(\mathbf{z}_u) \tag{17}$$

where β is the penalty strength and r is defined as

$$r(\mathbf{z}_u) := \mathbf{z}_u' \mathbf{\Pi}^{-\gamma} \mathbf{z}_u \tag{18}$$

where $\mathbf{\Pi} = \text{diag}(\boldsymbol{\pi})$ and γ is a non-negative hyperparameter. Notice that $r(\mathbf{z}_u)$ is larger if node u is assigned to a small community. Thus, nodes belonging to a small communities penalize the likelihood more than nodes belonging to large communities. The γ hyperparameter controls degree to which nodes are penalized (see Section 4.3).

With this penalized log-likelihood, the estimation procedure follows

$$\tilde{\mathbf{z}}_u^{(t+1)} \leftarrow \mathbf{z}_u^{(t)} - \text{E}\left(\frac{\partial \ell^2}{\partial \mathbf{z}_u \partial \mathbf{z}_u'} - \beta \frac{\partial r^2}{\partial \mathbf{z}_u \partial \mathbf{z}_u'} \right)^{-1} \left(\frac{\partial \ell}{\partial \mathbf{z}_u} - \beta \frac{\partial r}{\partial \mathbf{z}_u} \right) \Bigg|_{\mathbf{z}_u = \mathbf{z}_u^{(t)}} \tag{19}$$

for the node partition update. In order to incorporate the penalty into the estimation procedure, $\mathbf{\Pi}$ is estimated by

$$\hat{\mathbf{\Pi}} = \hat{\mathbf{\Pi}}(\mathbf{Z}) := \text{diag}(\hat{\boldsymbol{\pi}}(\mathbf{Z})) \tag{20}$$

which is simply the diagonalization of equation 8. Thus, the update rule evaluates to

$$\begin{aligned} \tilde{\mathbf{z}}_u^{(t+1)} &\leftarrow \mathbf{z}_u^{(t)} + \left(\mathbf{X}^{(t)\prime} \mathbf{W}_u^{(t)} \mathbf{X}^{(t)} + \beta \mathbf{\Pi}^{(t)^{-\gamma}} \right)^{-1} \left[\mathbf{X}^{(t)\prime} \mathbf{W}_u^{(t)} (\mathbf{a}_u - \mathbf{p}_u^{(t)}) - \beta \mathbf{\Pi}^{(t)^{-\gamma}} \mathbf{z}_u^{(t)} \right] \\ &= \left(\mathbf{X}^{(t)\prime} \mathbf{W}_u^{(t)} \mathbf{X}^{(t)} + \beta \mathbf{\Pi}^{(t)^{-\gamma}} \right)^{-1} \mathbf{X}^{(t)\prime} \mathbf{W}_u^{(t)} \mathbf{a}_u \end{aligned} \tag{21}$$

where $\mathbf{\Pi}^{(t)} = \hat{\mathbf{\Pi}}(\mathbf{Z}^{(t)})$. This is applied to all nodes at once using

$$\tilde{\mathbf{Z}}^{(t+1)} \leftarrow \left[\left(\mathbf{X}^{(t)\prime} \overline{\mathbf{W}}^{(t)} \mathbf{X}^{(t)} + \alpha \mathbf{\Pi}^{(t)^{-\gamma}} \right)^{-1} \mathbf{X}^{(t)\prime} \overline{\mathbf{W}}^{(t)} \mathbf{A} \right]' \tag{22}$$

to get the full partition update.

Before $\tilde{\mathbf{Z}}^{(t+1)}$ is projected onto the standard basis vectors, communities are dropped. This is done by first computing the size of each community in the partition update as $n_i^{(t)} = |\{u : \arg\max_j \tilde{z}_{uj}^{(t)} = i\}|$. The i^{th} column of $\tilde{\mathbf{Z}}^{(t)}$ is removed if $n_i^{(t)} < n_{\min}$ (for every i). We refer to this procedure by the function: $\text{drop}(\tilde{\mathbf{Z}}^{(t)}; n_{\min})$. The regularization term combined with the dropping procedure effectively reduces the number of communities from k_0 to a more optimal k_* over the course estimation.

For convenience, we allow the penalty term $\beta \mathbf{\Pi}^{-\gamma}$ to replace the smoothness regularization term $\alpha \mathbf{I}$ by setting $\alpha = c\beta$ where c is a scaling constant ensuring that $c\,\text{tr}(\mathbf{\Pi}^{-\gamma}) = \text{tr}(\mathbf{I})$. Note that when $\gamma = 0$, the penalty is equivalent to the the smoothness regularization. The full Fisher update procedure for this extension is

$$i.) \qquad \tilde{\mathbf{Z}}^{(t+1)} \leftarrow \mathbf{A}' \, \overline{\mathbf{W}}^{(t)} \mathbf{X}^{(t)} \left(\mathbf{X}^{(t)\prime} \overline{\mathbf{W}}^{(t)} \mathbf{X}^{(t)} + \alpha \mathbf{\Pi}^{(t)^{-\gamma}} \right)^{-1} \tag{23a}$$

$$ii.) \qquad \mathbf{Z}^{(t+1)} \leftarrow (\text{hardmax} \circ \text{drop})(\tilde{\mathbf{Z}}^{(t+1)}; \, n_{\min}) \tag{23b}$$

$$iii.) \qquad \mathbf{\Theta}^{(t+1)} \leftarrow \hat{\mathbf{\Theta}}(\mathbf{A}, \mathbf{Z}^{(t+1)}) \tag{23c}$$

This method eliminates the need to manually set the optimal number of communities. Instead, just the penalty strength α must be tuned while the minimum community size $n_{\min}$ can be set heuristically.

3.5 Extensions

Notice that the set of weight matrices $\{\mathbf{W}_u : u \in \mathcal{V}\}$ is made up of the inverse variances of each element a_{uv}. The estimation procedures presented above can be extended to graphs where it is assumed that a_{uv} is distributed according to a Poisson or Normal distribution by simply modifying the weight matrices. That is,

$$\begin{aligned}
a_{uv}|\mathbf{Z} \sim \text{Bern}(\mathbf{z}_u' \mathbf{\Theta} \mathbf{z}_v) \qquad &\Rightarrow \qquad \mathbf{W}_u(\mathbf{p}_u) = \text{diag}\left(\dots \frac{1}{p_{uv}(1-p_{uv})} \dots \right) \\
a_{uv}|\mathbf{Z} \sim \text{Pois}(\mathbf{z}_u' \mathbf{\Theta} \mathbf{z}_v) \qquad &\Rightarrow \qquad \mathbf{W}_u(\mathbf{p}_u) = \text{diag}\left(\dots \frac{1}{p_{uv}} \dots \right) \\
a_{uv}|\mathbf{Z} \sim \text{Norm}(\mathbf{z}_u' \mathbf{\Theta} \mathbf{z}_v, \, \sigma^2) \qquad &\Rightarrow \qquad \mathbf{W}_u(\mathbf{p}_u) = \text{diag}\left(\dots \frac{1}{\sigma^2} \dots \right)
\end{aligned}$$

Across each of these assumptions, the MLE of $\mathbf{\Theta}$ is the same.

Furthermore, this procedure can be extended to the Overlapping SBM — where nodes can belong to more than one community — by substituting the hardmax projection with the unit simplex projection and keeping the partition update rule the same. In this case, the $\hat{\mathbf{\Theta}}$ is an approximation of the MLE of $\mathbf{\Theta}$, not exact.

The Degree-Corrected SBM can also be estimated by substituting $p_{uv} = \mathbf{z}_u' \mathbf{\Theta} \mathbf{z}_v$ for $p_{uv} = \phi_u \phi_v \mathbf{z}_u' \mathbf{\Theta} \mathbf{z}_v$ where $\boldsymbol{\phi}$ adjusts probabilities for degree heterogeneity. This modification really only affects the computation of the weight matrices in the estimation procedure. The parameter $\boldsymbol{\phi}$ has a closed form MLE.

4 Analysis

4.1 Permutation Invariance

The local optimality of $\hat{\mathbf{Z}}$ and $\hat{\mathbf{\Theta}}$ is invariant to permutations of the community assignments. Let $\mathcal{R}$ be a set of $k \times k$ permutation matrices and let $\hat{\mathbf{Z}}$ and $\hat{\mathbf{\Theta}} = \hat{\mathbf{\Theta}}(\mathbf{A}, \hat{\mathbf{Z}})$ be estimators of the partition and structure matrices. Then the likelihood $\ell(\hat{\mathbf{Z}}, \hat{\mathbf{\Theta}})$ is permutation invariant; that is, $\forall\, \mathbf{R} \in \mathcal{R} \,:\, \ell(\hat{\mathbf{Z}}\mathbf{R}, \mathbf{R}\hat{\mathbf{\Theta}}\mathbf{R}') = \ell(\hat{\mathbf{Z}}, \hat{\mathbf{\Theta}})$.

Note that for any permutation $\mathbf{R} \in \mathcal{R}$, we have that $\mathbf{R}'\mathbf{R} = \mathbf{I}$. Therefore, for all $u, v \in \mathcal{V}$ we have $(\hat{\mathbf{z}}_u'\mathbf{R})(\mathbf{R}'\hat{\mathbf{\Theta}}\mathbf{R})(\mathbf{R}'\hat{\mathbf{z}}_v) = \hat{\mathbf{z}}_u'\mathbf{I}\hat{\mathbf{\Theta}}\mathbf{I}\hat{\mathbf{z}}_v = \hat{\mathbf{z}}_u'\hat{\mathbf{\Theta}}\hat{\mathbf{z}}_v$. It clearly follows then that $\ell(\hat{\mathbf{Z}}\mathbf{R}, \mathbf{R}'\hat{\mathbf{\Theta}}\mathbf{R}) = \ell(\hat{\mathbf{Z}}, \hat{\mathbf{\Theta}})$ meaning that any estimator — including a locally optimal one — is permutation invariant.

4.2 Relation to IRLS

...

4.3 Effects of γ and α

We now examine the effect of the γ hyperparameter as defined in section 3.4. Note that if $z_{ui} = 1$ and $z_{uj} = 0$ for $j \neq i$, then $r(\mathbf{z}_u) = \pi_i^{-\gamma}$. Thus for convenience, we define $r_i := \pi_i^{-\gamma}$ as the penalty associated with community i.

For $0 < \pi_i < 1$, note that $r_i \to \infty$ as $\pi_i \to 0$ and $r_i \to 1$ as $\pi_i \to 1$, which confirms that the penalty is larger for small community sizes. If $\pi_i > \pi_j \, (j \neq i)$, then r_j diverges exponentially faster than r_i does; consider that $\pi_i/\pi_j > 1$ so $r_j/r_i = (\pi_i/\pi_j)^\gamma \to \infty$ as $\gamma \to \infty$. From this we conclude that larger communities incur an exponentially greater penalty than smaller ones, and the magnitude of the difference is controlled by γ. Furthermore, consider

$$\frac{\partial\, r_i}{\partial\, \pi_i} = -\gamma\, \pi_i^{-(\gamma+1)}.$$

This implies that a small decrease in community size will increase the penalty by a factor proportional to γ. Therefore, the penalty is more sensitive when γ is large.

In practice, the penalty term leads to the appearance of $\mathbf{\Pi}$ in the Fisher matrix. When the i^{th} diagonal element of $\mathbf{\Pi}^{(t)}$ is large, it causes elements in the i^{th} column of $\tilde{\mathbf{Z}}^{(t+1)}$ to be small. This in turn makes it more likely that the i^{th} column will be dropped in the hardmax $\circ$ drop operation. Therefore, a larger value of γ leads to more aggressive dropping in the partition matrix.

If $\mathbf{\Pi}$ is rescaled so that $c\,\mathrm{tr}(\mathbf{\Pi}^{-\gamma}) = \mathrm{tr}(\mathbf{I})$, then the α hyperparameter is also relevant. In this case, γ controls the sensitivity of the penalty to community sizes and α controls its contribution to the objective function.

4.4 Time Complexity

A number of optimizations can be made to improve the time complexity of this technique. It is important to notice that the matrices $\mathbf{Z}$, $\mathbf{W}$, and $\mathbf{A}$ are all sparse. This means that full matrix multiplications are often not necessary. For example: $\mathbf{WX}$ can be computed by row-wise scalar-vector multiplication since $\mathbf{W}$ is diagonal; $\mathbf{Z\Theta}$ can be computed by simply indexing the columns of $\mathbf{\Theta}$ since $\mathbf{Z}$ is one-hot encoded; and $\mathbf{Z'AZ}$ can be computed with sparse matrix operations. The computation of $\overline{\mathbf{W}}$ can also be significantly improved by exploiting the block structure inherent to

$\mathbf{Z\Theta Z}'$. Additionally, the time to compute $(\mathbf{A}'\mathbf{WX})(\mathbf{X}'\mathbf{WX})^{-1}$ can be reduced significantly using numerical Cholesky decomposition solvers.

The total time complexity is ???. The major bottleneck comes from Cholesky decomposition, which scales with k. This method is very fast when the number of communities is small, but slows down with more communities.

It is also worth noting that, the full likelihood can be expressed as

$$\ell(\mathbf{Z}, \mathbf{\Theta}|\mathbf{A}) = \sum_{i,j} [m_{ij} \ln(\theta_{ij}) + (n_i n_j - m_{ij}) \ln(1 - \theta_{ij})]$$

which is convenient if the likelihood must be tracked during estimation.

5 Experiments

...

6 Conclusion

...

References